OLOS Protocol V3

Offline-First Transaction Integrity, Escrow-Bounded Authorization & Asynchronous Settlement

Status	Technical Validation Prototype
Version	OLOS-PROTOTYPE-0.3
Purpose	Architecture and protocol validation
Repository	OLOS V3 Prototype

Table of Contents

---

1. Overview

OLOS is a proposed transaction protocol designed to support secure transaction capture in disconnected environments while maintaining a path toward authoritative clearing and settlement when network connectivity is restored.

The architecture uses a **dual-layer strategy**:

1. Online synchronous settlement controls
2. Offline asynchronous risk management

The objective is not to assume that cryptography alone can prevent all offline double-spending scenarios.

Instead, OLOS combines:

- Cryptographically bound transaction identity

- Device-bound digital signatures
- Deterministic fixed-point monetary representation
- Long-term transaction replay detection
- Bounded offline authorization
- Merchant/device escrow allocation
- Local offline escrow decrementing
- Centralized clearing and reconciliation
- Atomic settlement state transitions
- Explicit security and fault semantics

> **Central architectural principle:** OLOS mitigates offline double-spending risk through bounded authorization and later authoritative reconciliation, while online transactions rely on synchronous settlement finality.

---

2. Design Goals

OLOS V3 is intended to explore whether the following architecture can provide a practical foundation for transaction processing where network connectivity is intermittent or unavailable.

Primary goals

- Support transaction creation while disconnected.
- Preserve transaction integrity during offline operation.
- Bind transactions to authorized devices.
- Represent monetary or asset values deterministically using fixed-point integers.
- Prevent replay of previously submitted transactions.
- Separate transaction identity from transport identity.
- Limit offline authorization exposure through escrow.
- Reconcile offline obligations after reconnection.
- Provide explicit transaction and security states.

- Support deterministic server-side verification.
- Provide an auditable transaction lifecycle.

Non-goals of this prototype

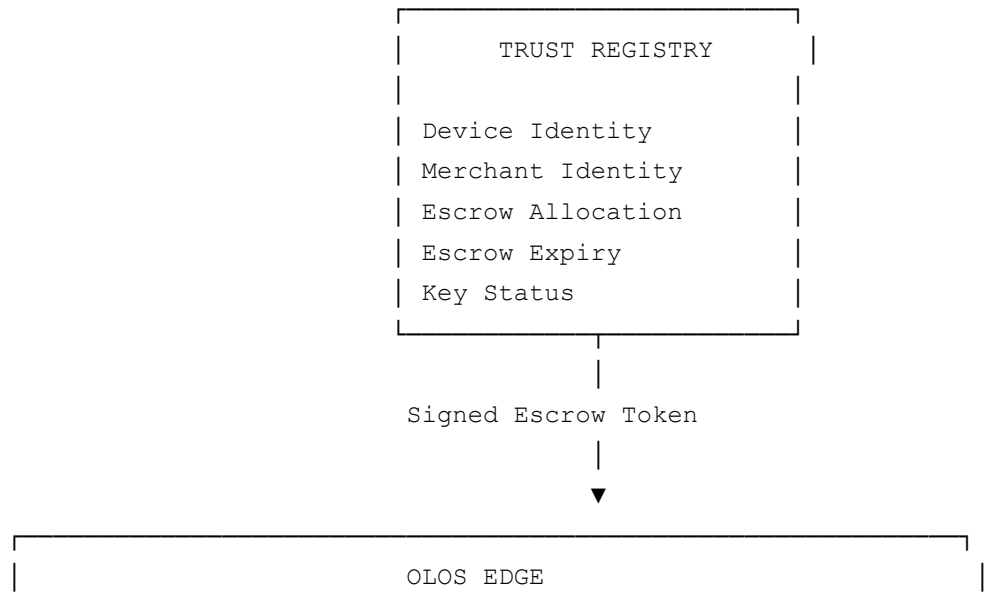
This repository does not claim to provide:

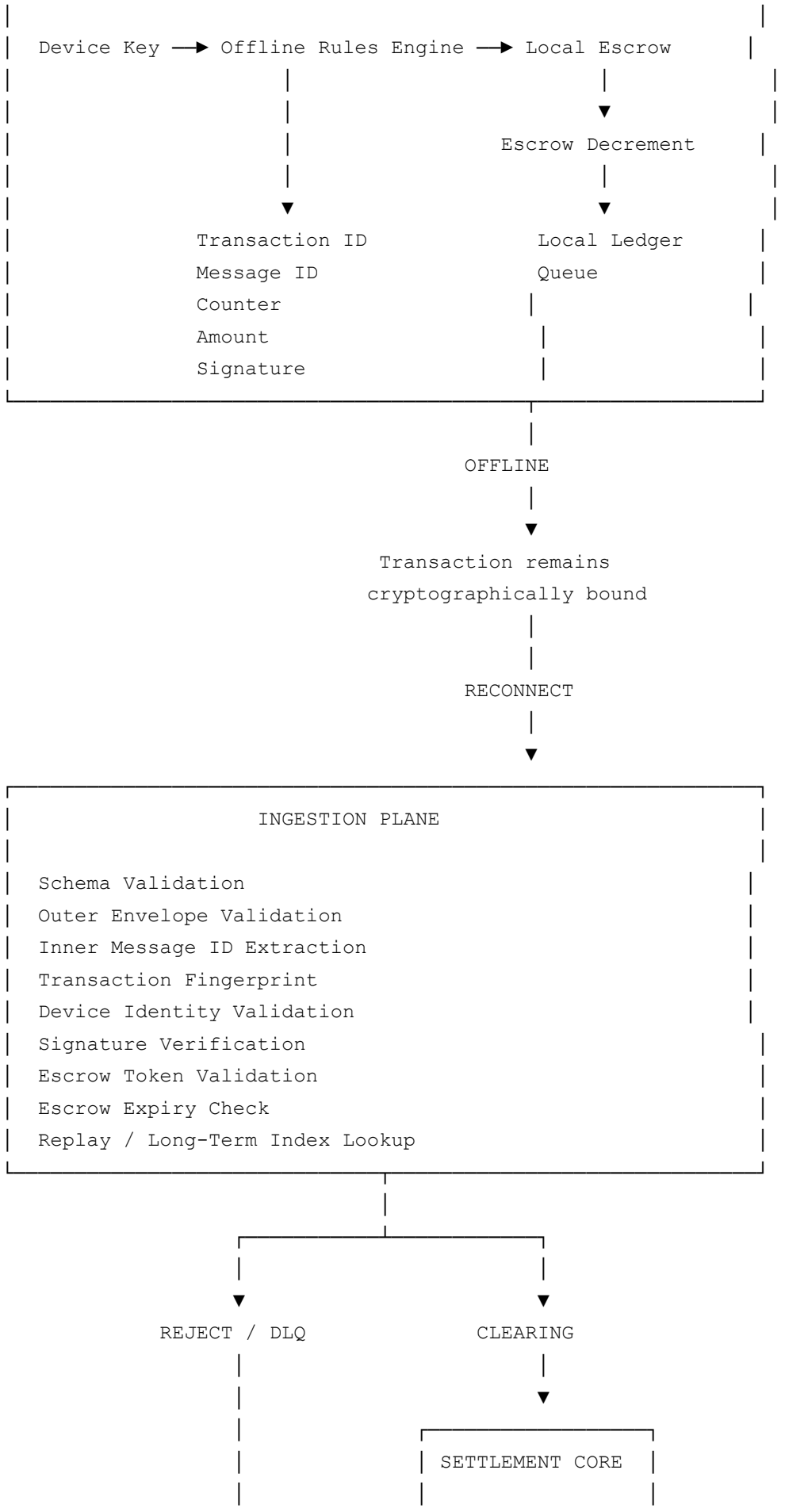
- PCI DSS certification
- EMV certification
- Payment-network certification
- Production-grade HSM integration
- Hardware attestation
- Regulatory approval
- Guaranteed elimination of all offline double-spending
- Production settlement finality
- Production-grade distributed consensus

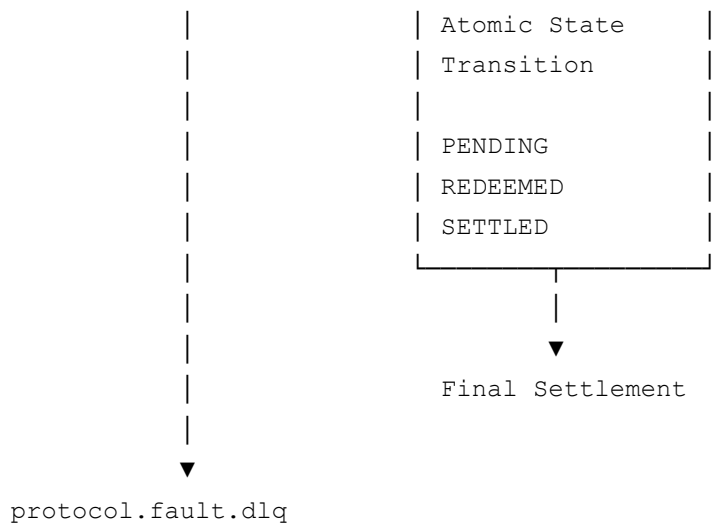
The prototype is intended to make the architecture concrete enough for review by payments, security, cryptography and distributed-systems specialists.

---

3. Core Architecture




Device Key
Offline Rules Engine
Local Escrow
Escrow Decrement
Transaction ID
Message ID
Counter
Amount
Signature
Local Ledger
Queue
OFFLINE
Transaction remains
cryptographically bound
RECONNECT
INGESTION PLANE
Schema Validation
Outer Envelope Validation
Inner Message ID Extraction
Transaction Fingerprint
Device Identity Validation
Signature Verification
Escrow Token Validation
Escrow Expiry Check
Replay / Long-Term Index Lookup
REJECT / DLQ
CLEARING
SETTLEMENT CORE



---

4. Online Double-Spending Mitigation

When terminals are connected to the network, OLOS assumes synchronous processing is available.

The conceptual flow is:

```
POS / Edge
    |
    ▼
Ingestion
    |
    ▼
Deduplication
    |
    ▼
Identity
    |
    ▼
Deterministic Compute
    |
    ▼
Settlement Core
    |
    ▼
Atomic State Transition
```

Online controls

Edge deduplication Incoming transaction messages can be checked against short-term caches and transaction identifiers to detect immediate duplicate submissions.

Deterministic compute Transaction and rules processing is intended to be deterministic so that identical inputs produce identical state transitions. The production architecture may use isolated execution environments such as MicroVMs for rules execution.

Settlement Core finality The authoritative settlement state tracks transaction or claim state. Example states: `PENDING`, `REDEEMED`, `SETTLED`.

If an already-consumed claim or balance cannot transition into a valid second redemption, the second attempt is rejected by the authoritative settlement layer.

> The critical security property is that the final state transition must be **atomic**.

---

5. Offline Double-Spending Mitigation

Offline processing creates a fundamentally different problem: the system cannot immediately know what another disconnected terminal has authorized.

OLOS therefore uses two complementary mechanisms:

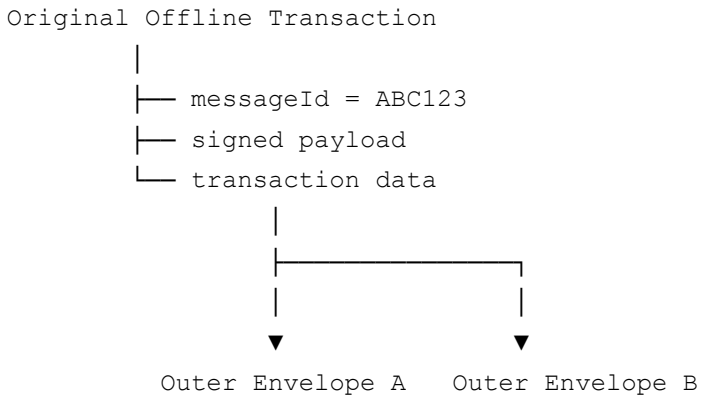
1. Transaction-level replay protection
2. Bounded offline authorization

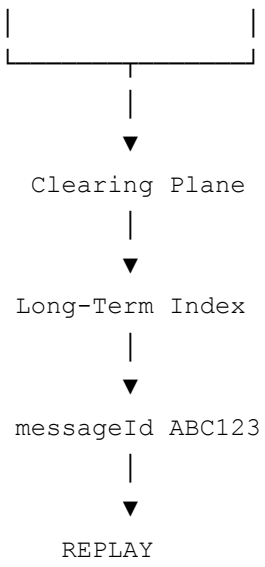
These address different threats.

---

6. Threat Vector A — Replay Attacks

Threat: An attacker or merchant forwards the same offline transaction more than once.





OLOS control

The inner transaction maintains its original identity and signature. The outer transport envelope does not replace the identity of the underlying transaction.

> A new outer transport envelope does not create a new economic transaction.

The Clearing Plane performs a long-term lookup against the inner transaction identifier. If the identifier has already been processed, the transaction is rejected.

Fault	`OLOS_ERR_REPLAY_ATTACK`
Event	`protocol.fault.double_spend_detected`
Routing	`protocol.fault.dlq`

---

7. Threat Vector B — Multi-Terminal Offline Over-Spending

This is the more difficult offline scenario.

```
Consumer balance = £50
Terminal A (offline)
    │
    └── Authorizes £50
Terminal B (offline)
    │
    └── Authorizes £50
```

```
Neither terminal can immediately see
the other's transaction.
```

OLOS addresses this through **bounded offline authorization**.

---

8. Escrow-Bound Offline Authorization

The Trust Registry provides authorized edge devices with a bounded escrow allocation.

```
Trust Registry
      │
      ▼
Terminal A
      │
      ├── Escrow: £200
      ├── Valid: 4 hours
      └── Signed authorization
```

The terminal may then authorize offline transactions only within that allowance.

```
Offline Escrow Allocation
─────────────────────────

Initial allocation:       £200
Transaction 1:            -£50
Remaining:                £150
Transaction 2:            -£75
Remaining:                 £75
Transaction 3:            -£75
Remaining:                   £0
Transaction 4:             -£10
                         REJECTED
```

The terminal therefore cannot create unlimited offline obligations.

Fault	`OLOS_ERR_ESCROW_EXCEEDED`
Event	`protocol.fault.escrow_exceeded`

---

9. Escrow Token

A conceptual escrow authorization contains:

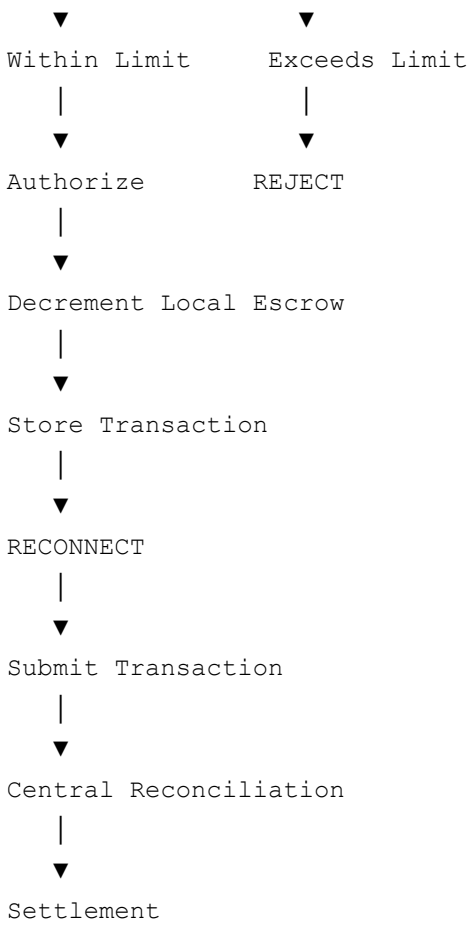
```
EscrowToken(
    escrow_id="escrow_001",
    merchant_id="merchant_789",
    device_id="terminal_001",
    asset="POINTS",
    authorized_amount=20000,
    remaining_amount=20000,
    valid_from=timestamp,
    valid_until=timestamp + 14400,
    token_version=1,
    signature=...
)
```

The production implementation would require a carefully designed authorization and key-management model. The prototype represents this as a cryptographically verifiable authorization object.

---

10. Escrow Lifecycle

```
ONLINE
   │
   ▼
Trust Registry
   │
   ▼
Escrow Issued
   │
   ▼
Terminal Receives Authorization
   │
   ▼
OFFLINE
   │
   ▼
Transaction Requested
   │
   ▼
Check Escrow
   │
   ├──────────────┐
   │              │
```



---

11. Transaction Identity vs Transport Identity

OLOS V3 explicitly separates:

Transaction identity

The transaction itself. Contains concepts such as:

- `messageId`
- `txn_id`
- **Merchant ID**
- **Device ID**
- **Amount**
- **Counter**
- **Timestamp**

- Transaction signature

Transport identity

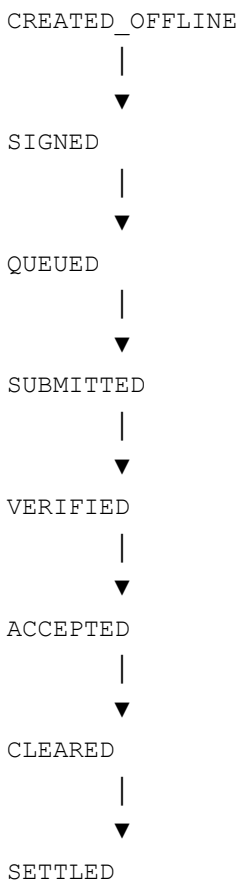
The network delivery wrapper. Contains concepts such as:

- Session identity
- Sequence number
- Reconnect timestamp
- Outer authentication
- Transport fingerprint

> **Design principle:** Transport retries must not create new transaction identities. This is fundamental to replay detection.

---

12. Transaction Lifecycle



Possible failure states

`REJECTED` · `REPLAYED` · `TAMPERED` · `REVOKED_DEVICE` · `ESCROW_EXCEEDED` · `ESCROW_EXPIRED` · `ESCROW_CONFLICT` · `COUNTER_ROLLBACK` · `DUPLICATE` · `SETTLEMENT_CONFLICT`

Account-level states (distinct from transaction states)

A concurrent multi-terminal over-spend (see OLOS-0011, Tier 3) is not a transaction failure — both offline transactions may be individually valid and signed. The conflict surfaces at the **account/identity** level, not the per-transaction level, so it requires its own state machine rather than an overloaded transaction status:

```
ACTIVE
   │
   ▼ (post-reconnection resolution detects negative balance)
QUARANTINED
   │
   ▼ (sufficient new earned points offset the negative debt)
RESTORED → ACTIVE
```

State	Meaning
`ACTIVE`	Normal standing; account may redeem within available balance/escrow.
`QUARANTINED`	Balance is negative following a resolved multi-terminal over-spend. Blinded `identityToken` is flagged network-wide. Redemptions are blocked; earning may continue.
`RESTORED`	Negative debt has been offset by new earned points; account transitions back to `ACTIVE`.

This state machine belongs to the **identity/account** domain, not the `transaction` domain — the individual transactions involved in the over-spend remain `SETTLED`; it is the consumer's account standing that changes.

---

13. Proposed OLOS V3 Components

```
OLOS V3
│
├── deterministic_math
│
├── identity
```

```
│
├── trust_registry
│
├── escrow
│
├── offline_ledger
│
├── transaction
│
├── transport
│
├── clearing_index
│
├── ingestion
│
├── settlement
│
├── reconciliation
│
└── security_tests
```

---

14. Component Responsibilities

deterministic_math

- Decimal input handling
- Fixed-point conversion
- Deterministic rounding
- Integer asset representation

The prototype uses `Decimal` and `ROUND_HALF_EVEN`.

```
£99.9950
    │
    ▼
Fixed-point scaling
    │
    ▼
Integer representation
```

identity

- Device identity
- Merchant binding
- Public-key registration
- Ed25519 signatures
- Device revocation
- Key versioning

`trust_registry`

- Device authorization
- Merchant authorization
- Escrow issuance
- Escrow status
- Device status

`escrow`

- Offline authorization limits
- Remaining offline allowance
- Expiration
- Asset type
- Device binding
- Merchant binding
- Escrow token validation

`offline_ledger`

- Local offline state
- Escrow decrement
- Offline transaction queue
- Local transaction persistence

transaction

- `messageId`
- `txn_id`
- Transaction fingerprint
- Digital signature
- Sequence counter
- Transaction state

transport

- Reconnection envelopes
- Session information
- Sequence numbers
- Outer authentication
- Transport fingerprinting

> Transport identity must remain separate from transaction identity.

clearing_index

- Long-term transaction lookup
- Replay detection
- Duplicate message detection
- Historical transaction identity

This is distinct from a short-lived ingestion cache.

ingestion

- Schema validation
- Transaction integrity
- Signature verification
- Device verification
- Merchant binding

- Escrow verification
- Replay checks
- Counter validation

settlement

- Authoritative transaction state
- Atomic state transitions
- Claim redemption
- Settlement finality

reconciliation

- Offline obligation reconciliation
- Escrow reconciliation
- Settlement conflicts
- Exception handling
- Clearing outcomes

---

15. Event Bus

```
protocol
 │
 ├── transaction
 │    ├── created
 │    ├── signed
 │    ├── forwarded
 │    ├── verified
 │    ├── accepted
 │    └── settled
 │
 ├── escrow
 │    ├── issued
 │    ├── decremented
 │    ├── exhausted
 │    ├── expired
 │    └── conflicted
```

```
│
└── fault
    ├── dlq
    ├── double_spend_detected
    ├── replay_attack
    ├── escrow_exceeded
    ├── escrow_expired
    └── escrow_conflict
```

---

16. Fault Codes

Code	Description
`OLOS_ERR_REPLAY_ATTACK`	The outer envelope is valid, but the inner transaction identity already exists in the historical clearing index.
`OLOS_ERR_ESCROW_EXCEEDED`	The edge terminal attempted to authorize an offline transaction beyond its available escrow allowance.
`OLOS_ERR_ESCROW_EXPIRED`	The terminal attempted to use an escrow authorization after its validity window.
`OLOS_ERR_ESCROW_INVALID`	—
`OLOS_ERR_ESCROW_CONFLICT`	The reconciliation process identifies a conflict between offline obligations and authoritative escrow state.
`OLOS_ERR_DEVICE_REVOKED`	—
`OLOS_ERR_INVALID_SIGNATURE`	—
`OLOS_ERR_COUNTER_ROLLBACK`	—
`OLOS_ERR_DUPLICATE_MESSAGE`	—
`OLOS_ERR_SETTLEMENT_CONFLICT`	The transaction cannot transition atomically into the requested settlement state.

---

17. Security Model

OLOS V3 separates four security questions:

Question	Description
Authenticity	Did an authorized device sign the transaction?
Integrity	Was the transaction modified after signing?
Authorization	Was the device authorized to create the transaction while offline?
Settlement Safety	Can the transaction be accepted and settled within the risk limits defined by the system?

These are deliberately separate properties. A valid digital signature alone does not answer all four questions.

---

18. Double-Spending Model

OLOS V3 uses a layered mitigation strategy.

Scenario	Primary Control
Duplicate online submission	Ingestion deduplication
Duplicate transaction replay	Long-term message ID index
New transport envelope around old transaction	Inner transaction identity
Online balance double spend	Atomic Settlement Core
Offline terminal over-authorization	Escrow limit
Offline escrow expiration	Escrow validity period
Compromised/revoked device	Device registry
Modified transaction	Digital signature
Counter rollback	Monotonic sequence validation
Settlement collision	Atomic state transition
Offline reconciliation conflict	Reconciliation engine

---

19. Critical Security Principle

OLOS V3 should **not** be described as providing unlimited offline double-spend prevention.

The more precise statement is:

> OLOS V3 mitigates offline double-spending risk by limiting the amount of value that an edge device can authorize while disconnected and by ensuring that each transaction maintains a persistent cryptographic identity for later reconciliation and replay detection.

This means the system's offline exposure is **bounded by policy**.

```
Maximum Offline Economic Exposure
=
Authorized Escrow
×
Number of Authorized Devices
×
Potential Compromise Window
```

The production protocol must define how those parameters are controlled.

---

20. Security Test Suite

The V3 prototype should test at least:

Transaction integrity

- Valid transaction accepted
- Modified amount rejected
- Modified merchant rejected
- Modified device ID rejected
- Invalid signature rejected

Replay

- Same envelope replay rejected
- Same transaction in new envelope rejected

- Duplicate message ID rejected
- Long-term replay detected

Escrow

- Valid escrow accepted
- Escrow decremented
- Escrow exhausted
- Escrow exceeded
- Expired escrow rejected
- Invalid escrow signature rejected
- Wrong device rejected
- Wrong merchant rejected

Device security

- Unknown device rejected
- Revoked device rejected
- Invalid device key rejected
- Counter rollback rejected

Settlement

- Valid transaction settled
- Already redeemed claim rejected
- Duplicate redemption rejected
- Settlement conflict detected

Concurrency

- Concurrent duplicate submission
- Multiple reconnect attempts
- Multiple transport envelopes
- Race conditions during settlement

Persistence

- Transaction uniqueness
- Message ID uniqueness
- Escrow state persistence
- Audit event persistence
- Atomic database updates

---

21. Example End-to-End Scenario

1. Terminal is online.
2. Trust Registry authorizes: Terminal A, Merchant 789, £200 offline escrow, valid for 4 hours.
3. Terminal loses network connectivity.
4. Customer presents transaction for £50.
5. Terminal verifies: device authorized, escrow valid, £50 ≤ £200.
6. Transaction is signed.
7. Local escrow becomes £150 remaining.
8. Transaction is stored locally.
9. Terminal reconnects.
10. Outer transport envelope is created.
11. Clearing Plane extracts inner `messageId`.
12. Long-term index confirms `messageId` not previously processed.
13. Signature is verified.
14. Escrow authorization is verified.
15. Transaction is accepted.
16. Escrow obligation is reconciled.
17. Settlement Core performs atomic state transition.

18. **Transaction becomes** `SETTLED`.

19. `messageId` **is permanently indexed.**

20. **A second submission of the same transaction is rejected:** `OLOS_ERR_REPLAY_ATTACK`.

---

22. Example Replay Scenario

```
Transaction:
    messageId = TX123
    Signature = SIGABC

First submission:
    TX123 → ACCEPTED

Second submission:
    New outer envelope
    New session
    New transport fingerprint

Inner transaction:
    messageId = TX123
    Signature = SIGABC

Long-Term Index:
    TX123 already exists

Result:
    REJECT

Fault:
    OLOS_ERR_REPLAY_ATTACK
```

The new outer envelope does not create a new transaction.

---

23. Example Escrow Exhaustion

```
Escrow:
    £200

Transaction A:
    £100
```

```
    Remaining = £100

Transaction B:
    £75
    Remaining = £25

Transaction C:
    £25
    Remaining = £0

Transaction D:
    £10

Result:
    REJECT

Fault:
    OLOS_ERR_ESCROW_EXCEEDED
```

---

24. V3 Prototype Limitations

The following areas require specialist review before any production implementation.

Secure hardware The prototype does not prove that a terminal private key cannot be extracted. A production system may require Secure Elements, HSMs, Trusted Execution Environments, hardware-backed key storage, and device attestation.

Device cloning A production design must address what happens if a legitimate terminal's credentials are cloned.

Counter management Multiple devices and terminal restarts create difficult state-management questions.

Escrow allocation The Trust Registry must define how escrow is allocated, revoked and reconciled.

Escrow conflict The system needs an explicit policy for what happens if offline obligations exceed the authoritative state.

Regulatory compliance A production implementation would require assessment against relevant payment, financial and data-security requirements.

---

25. Recommended V3 Research Questions

The primary purpose of the prototype is to enable expert validation. The following questions should be reviewed by payments-security specialists:

1. Does the separation between inner transaction identity and outer transport identity provide sufficient replay protection when combined with a long-term clearing index?
2. Is escrow-bounded offline authorization a viable mechanism for limiting economic exposure created by disconnected terminals?
3. What additional controls are required to prevent or detect terminal cloning, escrow-token duplication, local ledger rollback or malicious offline devices?
4. How should escrow conflicts be resolved when multiple disconnected devices create obligations that cannot all be honored?
5. What hardware-backed security model would be required for production deployment?
6. How would this architecture map onto existing payment, wallet, loyalty, stored-value or digital-asset systems?
7. What parts of the design would require formal verification or independent cryptographic review?

---

26. V3 Development Roadmap

Phase 1 — Protocol Prototype

- Deterministic monetary math
- Device identity
- Ed25519 signatures
- Transaction identity
- Transport identity
- Escrow tokens
- Local escrow decrement
- Replay index

- Ingestion validation
- Atomic settlement simulation

Phase 2 — Adversarial Testing

- Replay attacks
- Device cloning simulation
- Counter rollback
- Database rollback
- Escrow duplication
- Escrow expiry
- Concurrent submissions
- Reconciliation conflicts

Phase 3 — Security Review

Engage independent:

- Payments security engineers
- Cryptographers
- Distributed systems engineers
- Hardware security specialists
- Payment-network architects

Phase 4 — Controlled Demonstration

```
ONLINE
   ↓
ESCROW ISSUED
   ↓
NETWORK LOST
   ↓
OFFLINE TRANSACTIONS
   ↓
ESCROW DECREMENT
   ↓
RECONNECT
   ↓
```

```
REPLAY DETECTION
   ↓
CLEARING
   ↓
ATOMIC SETTLEMENT
   ↓
AUDIT
```

Phase 5 — Production Feasibility

Investigate:

- Hardware security
- Key management
- Device provisioning
- Attestation
- Distributed databases
- High availability
- Settlement integration
- Compliance
- Operational controls

---

27. Project Status

OLOS V3 is currently a technical validation concept and prototype architecture.

The purpose of the project is to determine whether the proposed combination of:

Cryptographic Transaction Identity + Long-Term Replay Detection + Bounded Offline Escrow + Asynchronous Reconciliation + Atomic Settlement

can form a practical architecture for transaction processing in environments where connectivity is intermittent or unavailable.

The next milestone is independent technical validation.

---

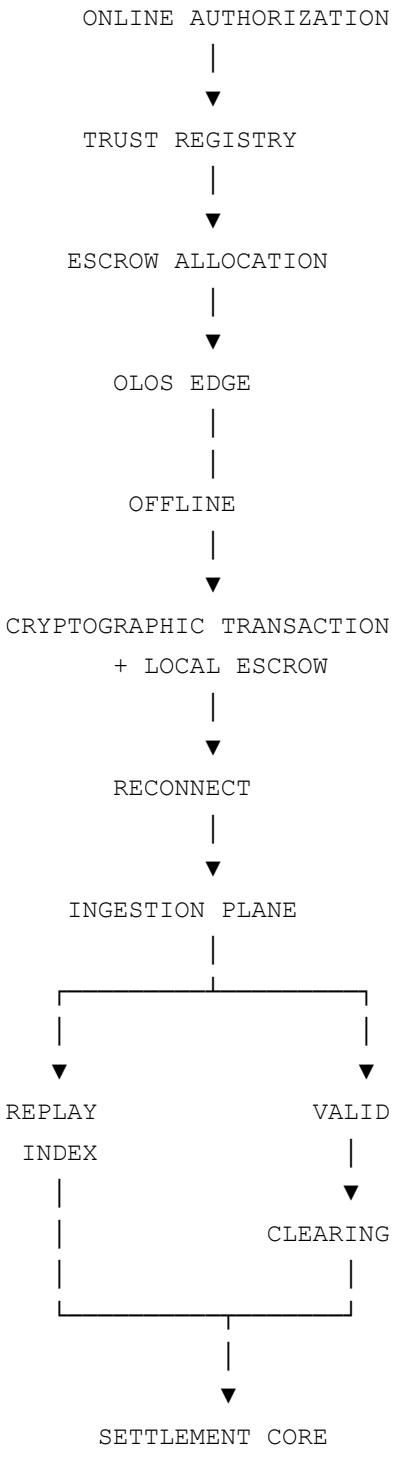
28. Core Proposition

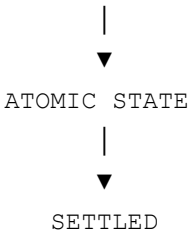
> Can transaction systems safely extend authorization into disconnected environments by combining cryptographically verifiable transaction identity with bounded offline economic exposure and authoritative asynchronous reconciliation?

OLOS V3 is intended to make that proposition testable.

---

29. Final Architecture Summary





OLOS V3 — One-Sentence Summary

OLOS is a proposed offline-first transaction architecture that combines cryptographically bound transaction identity, bounded escrow-based offline authorization, long-term replay detection and authoritative asynchronous settlement to limit and reconcile the risks created by disconnected transaction processing.